Experiment Number : 13 **Date: 03-03-26**

Theeran

24BCS298

CSE-A1

Implementation of User-defined Exception in Java

Aim:

Write a java program to implement User-defined Exception.

PRE LAB EXERCISE

QUESTIONS

1. What is Java custom exception?

- A **custom exception** in Java is a user-defined exception created by extending the `Exception` or `RuntimeException` class.
- It is used to define application-specific error conditions that are not covered by Java's built-in exceptions.

2. What are the two types of custom exceptions in Java?

There are two types:

1 Checked Custom Exception

- Created by extending `Exception`
- Must be handled using `try-catch` or declared using `throws`
- Checked at compile time

Example:

```java
class MyException extends Exception {
}
```

2️⃣ Unchecked Custom Exception

- Created by extending `RuntimeException`
- Not mandatory to handle
- Occurs at runtime

Example:

```java
class MyException extends RuntimeException {
}
```

IN LAB EXERCISE

Objective

To understand and implement Checked and Unchecked custom exceptions.

Source Code

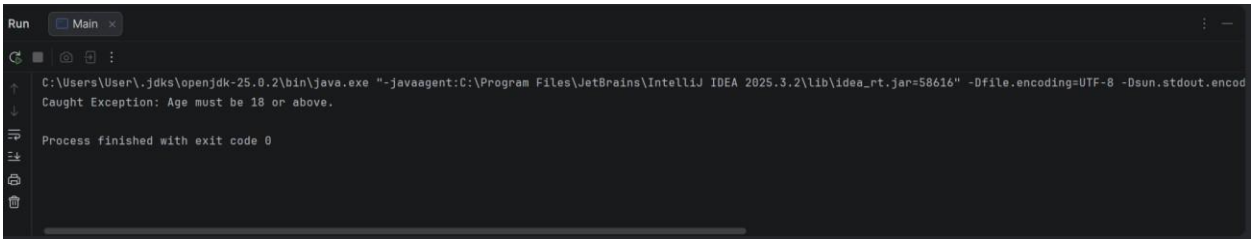
Ex 1: // Custom Checked Exception

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String m) {
        super(m);
    }
}
// Using the Custom Exception
public class AgeCheck{
    public static void validate(int age)
      throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
        }
        System.out.println("Valid age: " + age);
    }
public static void main(String[] args) {
  try {
```

```
            validate(12);
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

Output 1

Caught Exception: Age must be 18 or above.



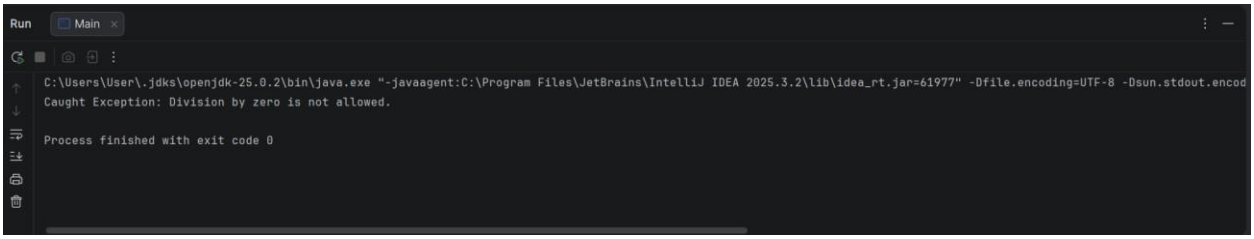
Ex 2: // Custom Unchecked Exception

```
class DivideByZeroException extends RuntimeException {
    public DivideByZeroException(String m) {
        super(m);
    }
}
// Using the Custom Exception
public class TestSample {
    public static void divide(int a, int b) {
        if (b == 0) {
            throw new DivideByZeroException("Division by zero is not allowed.");
        }
        System.out.println("Result: " + (a / b));
    }
```

```
public static void main(String[] args) {
  try {
        divide(10, 0);
      } catch (DivideByZeroException e) {
        System.out.println("Caught Exception: " + e.getMessage());
      }
   }
}
```

Output 2

Caught Exception: Division by zero is not allowed.



POST LAB EXERCISE

1. What is required to create a custom exception in Java?
   A. Extend the Object class
   B. Extend the Throwable class
   C. Extend Exception or RuntimeException
   D. Implement the Serializable interface

2. List some use cases for the checked and unchecked exceptions.

Checked Exceptions (Compile-time exceptions)
(Must be handled using try-catch or throws)
Use cases:

- File handling errors (IOException)
- Database connection errors (SQLException)
- Network communication errors
- Class loading issues (ClassNotFoundException)
- Validation in business logic where recovery is possible

✓ Used when the programmer is expected to handle the error.

Unchecked Exceptions (Runtime exceptions)

(Not mandatory to handle)

Use cases:

- Logical programming errors
- Division by zero (ArithmeticException)
- Null reference access (NullPointerException)
- Invalid array index (ArrayIndexOutOfBoundsException)
- Invalid method arguments

✓ Used when the error is due to programming mistakes and cannot normally be recovered at runtime.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		